Universidad de Caldas

Ingeniería de Sistemas y Computación

Análisis y Diseño de Algoritmos

Manual técnico de la solución base

Proyecto Oráculo (ADAG1), Grupo G01

Primera entrega

Autores:

Andrés Santiago Restrepo Duque

Keyla Daniela Cartagena Díaz

Manizales

Noviembre 4 de 2026

Contenido

1. Introducción **4**

1.1. Propósito del documento 4

1.2. Alcance de la solución base 4

1.3. Convenciones 4

2. Entorno de ejecución **4**

2.1. Plataforma y hardware 4

2.2. Dependencias de software 5

2.3. Recursos externos 6

2.4. Consideraciones de la plataforma 6

3. Arquitectura **6**

3.1. Estructura del repositorio 6

3.2. Responsabilidad de los módulos 7

3.3. Interfaces utilizadas 8

3.4. Flujo entre componentes 8

4. Formatos de datos **9**

4.1. Instancia 9

4.2. Configuración 9

4.3. Resultado del oráculo 9

4.4. Archivo entrega.json 10

4.5. Archivos de evidencia 10

5. Algoritmo de búsqueda **11**

5.1. Notación 11

5.2. Vecindad por coordenadas 11

5.3. Criterio de poda basado en trazas 12
5.4. Pseudocódigo 13
5.5. Desempate, puntos iniciales y parada 14
5.6. Propiedades 14
5.7. Análisis de complejidad 15
Número de consultas 15
Costo de una consulta 15
Trabajo de la heurística 16
Costo total y comparación 16
Memoria 16
5.8. Estructuras de datos 17

6. Procedimiento de ejecución **17**
6.1. Notebook de búsqueda 17
6.2. Notebook de evaluación 18
6.3. Reanudación después de una desconexión 18
6.4. Parámetros 19

7. Interpretación de las salidas **20**

8. Verificación **20**
8.1. Comprobaciones internas 20
8.2. Prueba con oráculo sintético 21
8.3. Prueba mínima con el modelo 21
8.4. Reproducibilidad de los resultados 22

9. Supuestos, limitaciones y trabajo pendiente **22**
9.1. Supuestos 22
9.2. Limitaciones y errores conocidos 22

9.3. Componentes no desarrollados 23

Referencias **23**

1. Introducción

1.1. Propósito del documento

Este manual describe la construcción y el funcionamiento de la solución base del Proyecto Oráculo, de manera que una persona ajena al equipo pueda instalarla, ejecutarla, interpretar sus resultados y reproducir la configuración reportada en entrega.json sin explicaciones adicionales de los autores. Complementa el informe académico de la primera entrega, que contiene la formalización del problema y la demostración de NP-dureza; aquí se documenta la implementación.

1.2. Alcance de la solución base

La solución base resuelve el problema de seleccionar una configuración de prompt, es decir, un índice para cada una de las cinco ranuras del catálogo (rol, estrategia, formato, estilo y cierre) junto con una temperatura de generación, que maximice la precisión del oráculo sobre las 100 instancias de búsqueda. La precisión de una configuración es la fracción de instancias en las que la respuesta del modelo cumple todas sus restricciones verificables.

El método implementado es un ascenso por coordenadas con reinicios, al que se añade una regla de poda que utiliza las trazas de fallos del oráculo para descartar opciones. La solución entrega la mejor configuración observada durante la búsqueda; no se afirma que sea un óptimo global. Las estrategias de reparto de presupuesto del enunciado general, la exploración de temperaturas distintas de 0.0 y la validación experimental del crecimiento de $T(n)$ quedan fuera de esta versión y se listan en la sección 9.3.

1.3. Convenciones

Los nombres de archivos, funciones y variables se escriben tal como aparecen en el código. Las rutas de la forma /content/. . . corresponden al sistema de archivos de Google Colab. Las cantidades en cursiva se definen en la sección 5.1 y $|X|$ denota la cardinalidad del conjunto X.

2. Entorno de ejecución

2.1. Plataforma y hardware

La generación de respuestas requiere una GPU compatible con CUDA. La solución se desarrolló y probó en Google Colab con una GPU NVIDIA T4 de 16 GB. La función cargar_modelo de ayudas.py interrumpe la ejecución con un mensaje explícito si no detecta una GPU.

Tabla 1. Requisitos de plataforma.

Elemento	Requisito
Plataforma	Google Colab (entorno de ejecución con GPU)
GPU	NVIDIA T4, arquitectura sm_75, 16 GB de VRAM
Sistema operativo	Linux de Colab (Ubuntu); no se probó en Windows ni macOS
Python	3.10 o superior (versión provista por Colab)
Modelo de lenguaje	Qwen/Qwen3-1.7B (alias qwen17b), pesos en float16, aproximadamente 3,4 GB de VRAM
Almacenamiento	Google Drive, opcional, para conservar la caché entre sesiones

2.2. Dependencias de software

Las dependencias se declaran en requirements.txt y se instalan en la primera celda de cada notebook. La tabla 2 indica la función de cada una dentro de la solución.

Tabla 2. Dependencias y su uso.

Paquete	Versión	Uso en la solución
torch	la de Colab	Inferencia del modelo en GPU
transformers	última	Carga del tokenizador y del modelo, generación de texto
accelerate	última	Ubicación del modelo en el dispositivo (device_map)
bitsandbytes	$\geq 0.46.1$	Carga de modelos cuantizados a 4 bits (alias alternativos)
nltk, spacy, emoji, langdetect, immutabledict	última	Requeridos por los verificadores de restricciones de open-instruct
pandas, matplotlib	la de Colab	Tablas de evidencia y curva de búsqueda
tqdm	la de Colab	Barra de progreso del oráculo

2.3. Recursos externos

Además de los paquetes de Python, la ejecución descarga los siguientes recursos: el repositorio allenai/open-instruct, clonado en /content/open-instruct, que contiene el registro de verificadores IFEvalG usado por el oráculo; el modelo en_core_web_sm de spaCy; los paquetes punkt, punkt_tab, averaged_perceptron_tagger y averaged_perceptron_tagger_eng de nltk, que se descargan manualmente porque el descargador de nltk falla detrás del proxy de Colab; y los pesos del modelo desde Hugging Face, sin autenticación.

2.4. Consideraciones de la plataforma

- Después de la instalación es obligatorio reiniciar la sesión de Colab. Si no se reinicia, transformers puede quedar parcialmente actualizado y cargar_modelo falla con un error poco descriptivo.
- La T4 no ofrece bfloat16 nativo; ayudas.py fuerza float16 como tipo de cómputo.
- Las sesiones gratuitas de Colab se desconectan por inactividad o por tiempo de uso. La caché del oráculo en Google Drive permite continuar una búsqueda interrumpida sin repetir generaciones (sección 6.3).
- El repositorio debe ser público para que los notebooks puedan clonarlo desde Colab.

3. Arquitectura

3.1. Estructura del repositorio

La figura 1 muestra la organización de carpetas. Los archivos oraculo.py, ayudas.py, datos_visibles.json y proyecto_oraculo.ipynb provienen del repositorio del curso y no se modifican.

```
ADA-2026-2-GP01_Andres_y_Keyla/
├── notebooks/
│   ├── 01_busqueda.ipynb ..... búsqueda, trazas, validación y entrega.json
│   └── 02_evaluacion.ipynb ..... evaluación de entrega.json
├── resultados/ ..... evidencia generada por 01_busqueda.ipynb
│   ├── evaluaciones.csv, registro_busqueda.json
│   ├── derrotas.csv, poda.json, curva_busqueda.png
│   └── cache_oraculo.json.gz
├── pruebas/
│   └── prueba_busqueda_simulada.py ..... prueba sin GPU
├── docs/ ..... manual técnico
├── entrega.json ..... configuración entregada
├── oraculo.py, ayudas.py, datos_visibles.json ..... archivos del curso
├── proyecto_oraculo.ipynb ..... notebook original del curso
└── requirements.txt, README.md, .gitignore
```

Figura 1. Estructura de carpetas del repositorio.

3.2. Responsabilidad de los módulos

Tabla 3. Módulos y responsabilidades.

Módulo	Responsabilidad
oraculo.py	Arma el prompt de una configuración para cada instancia, genera la respuesta con el modelo, la verifica con open-instruct y guarda el resultado en caché. Se utiliza como caja negra.
ayudas.py	Carga el modelo y los datos, separa las particiones de búsqueda y validación, dibuja la curva de búsqueda y escribe entrega.json.
01_busqueda.ipynb	Implementa la heurística, registra cada evaluación, aplica la poda basada en trazas, valida la configuración elegida y exporta la evidencia.
02_evaluacion.ipynb	Comprueba el formato de entrega.json y mide su precisión sobre los datos de prueba o, en su ausencia, sobre la partición de validación.
prueba_busqueda_simulada.py	Ejecuta las funciones del notebook de búsqueda contra un oráculo sintético y verifica las propiedades de la sección 5.6.

3.3. Interfaces utilizadas

La solución solo usa las interfaces públicas del código del curso, resumidas en la tabla 4. Del catálogo de opciones se usa únicamente el número de opciones por ranura y el texto de una opción cuando se muestra la evidencia de una poda.

Tabla 4. Interfaces públicas utilizadas.

Interfaz	Entrada	Salida
Oraculo(modelo, datos, validacion, cache, lote, presupuesto)	Modelo cargado, particiones, ruta de caché y parámetros de lote	Objeto oráculo
oraculo.evaluar(config, instancias, semilla)	Configuración, lista de instancias, semilla	Resultado con precision, trazas y n
oraculo.validar(config, n, semilla)	Configuración y tamaño de muestra (None = todas)	Resultado sobre la partición de validación; imprime las familias más falladas
cargar_modelo(alias)	Alias del modelo	Objeto Modelo (red, tokenizador, nombre)
cargar_datos(ruta), dividir(datos)	Ruta del JSON; lista de instancias	Lista de instancias; tupla (busqueda, validacion)
entrega(grupo, config, semana, ruta)	Identificador, configuración, semana, ruta	Escribe entrega.json

3.4. Flujo entre componentes

El notebook de búsqueda llama a ayudas.py para obtener el modelo y las particiones, construye un objeto Oraculo y, en cada paso de la heurística, invoca oraculo.evaluar. El oráculo consulta primero su caché; solo genera respuestas para los pares de configuración e instancia que no estén guardados, y las verifica con el registro de open-instruct. La heurística recibe la precisión y las trazas, actualiza sus estructuras (sección 5.8) y decide la siguiente configuración. Al terminar, el notebook llama a oraculo.validar y a entrega. El notebook de evaluación solo lee entrega.json y llama a oraculo.evaluar una vez.

4. Formatos de datos

4.1. Instancia

Los archivos datos_visibles.json y datos_prueba.json contienen una lista JSON de instancias con los campos de la tabla 5.

Tabla 5. Campos de una instancia.

Campo	Tipo	Obligatorio	Descripción
id	entero	sí	Identificador único de la instancia
prompt	texto	sí	Petición completa al modelo, con sus restricciones redactadas
ids	lista de texto	sí	Familias de restricción de open-instruct, por ejemplo keywords:existence
kwargs	lista de objetos	sí	Parámetros de cada restricción, en el mismo orden que ids
restriccion	texto	no	Restricciones separadas por tabulador; la usa la opción de cierre que las repite
split	texto	no	busqueda, validacion o descartada

4.2. Configuración

Una configuración es un objeto con seis llaves. Las cinco ranuras toman un entero en $\{0, 1, \ldots, 7\}$ y la temperatura toma un valor en $\{0.0, 0.3, 0.7\}$. Con la temperatura fija en 0.0 hay $8^5 = 32\,768$ configuraciones; con las tres temperaturas, 98 304. La solución base trabaja con temperatura 0.0. Por ejemplo:

```
{"rol": 0, "estrategia": 0, "formato": 0, "estilo": 0, "cierre": 0, "temperatura": 0.0}
```

4.3. Resultado del oráculo

evaluar y validar devuelven un objeto con tres atributos: precision, número real en $[0, 1]$; n, número de instancias evaluadas; y trazas, lista que contiene solo las instancias falladas. Cada traza tiene id (instancia), violo (primera familia de restricción que no se cumplió) y salida (texto generado). Como la evaluación es de todo o nada, una instancia con varias restricciones incumplidas aparece una sola vez, con la primera que falló.

4.4. Archivo entrega.json

Contiene el identificador del grupo, la configuración elegida y la semana académica. Lo genera la función entrega al final del notebook de búsqueda y lo lee el notebook de evaluación:

{"grupo": "G01", "config": {. . . }, "semana": 5}

4.5. Archivos de evidencia

Tabla 6. Archivos generados en resultados/.

Archivo	Contenido
evaluaciones.csv	Una fila por consulta nueva: número de consulta, reinicio, pasada, ranura modificada, opción, los cinco índices, precisión, aciertos y segundos
registro_busqueda.json	Parámetros, resultado de cada reinicio, mejor configuración, precisión de búsqueda y de validación, fallos de validación por familia, opciones vivas al final y, por cada consulta, las familias violadas y los ids de las instancias falladas
derrotas.csv	Una fila por derrota observada: ranura, opción, vecindario, opción ganadora, precisiones, instancias perdidas y ganadas, reinicio y pasada
poda.json	Una entrada por opción descartada: ranura, opción, número de vecindarios, pérdida neta acumulada, evidencia de cada vecindario, familias violadas en las instancias perdidas y un extracto de la salida del modelo
curva_busqueda.png	Precisión de cada consulta y mejor precisión acumulada
cache_oraculo.json.gz	Copia comprimida de la caché del oráculo; permite rehacer el análisis sin generar de nuevo

5. Algoritmo de búsqueda

5.1. Notación

Tabla 7. Notación utilizada.

Símbolo	Significado
m	Número de ranuras; $m = 5$
K	Opciones por ranura; $K = 8$
I	Conjunto de instancias de búsqueda; $n = \|I\| = 100$
c	Configuración; $c[j]$ es la opción elegida en la ranura j
$A(c) \subseteq I$	Instancias que c resuelve (cumplen todas sus restricciones)
$F(c) = I \setminus A(c)$	Instancias que c falla
$v_c(i)$	Familia violada por c en la instancia $i \in F(c)$, según la traza
$\mathrm{Acc}(c) = \|A(c)\|/n$	Precisión de c sobre la búsqueda
V_j	Opciones vivas de la ranura j (no podadas)
R, P	Número de reinicios y máximo de pasadas por reinicio
$L(o, b)$, $G(o, b)$	Instancias que o pierde y gana frente a b (sección 5.3)
$D(j, o)$	Vecindarios en los que o fue derrotada en la ranura j
τ, κ, μ	Pérdida neta mínima para una derrota, vecindarios con derrota necesarios para podar y mínimo de opciones vivas por ranura

5.2. Vecindad por coordenadas

Dada una configuración c y una ranura j, la vecindad de c en la coordenada j es el conjunto de configuraciones que difieren de c solo en esa ranura y cuya opción está viva o es la opción vigente:

$$N_j(c) = \{\, c' : c'[h] = c[h] \text{ para todo } h \neq j,\ \ c'[j] \in V_j \cup \{c[j]\} \,\}.$$

Un paso del método evalúa toda la vecindad $N_j(c)$ y reemplaza c por el elemento de mayor precisión. Una pasada aplica este paso a las ranuras en el orden rol, estrategia, formato, estilo y cierre.

5.3. Criterio de poda basado en trazas

Sea b la opción ganadora de la ranura j y o otra opción evaluada en el mismo paso. Las configuraciones c_b y c_o coinciden en todas las ranuras excepto en j, de modo que cualquier diferencia entre sus resultados se atribuye al cambio de opción. Se definen las instancias perdidas y ganadas por o frente a b:

$$L(o, b) = A(c_b) \cap F(c_o), \qquad G(o, b) = A(c_o) \cap F(c_b),$$

y la pérdida neta

$$\Delta(o, b) = |L(o, b)| - |G(o, b)|.$$

Se llama vecindario del paso a la tupla u de índices de las otras cuatro ranuras. Si $\Delta(o, b) \geq \tau$, se registra una derrota de o en el vecindario u. Sea $D(j, o)$ el conjunto de vecindarios distintos en los que o ha sido derrotada en la ranura j. La opción o se elimina de V_j cuando se cumplen las dos condiciones siguientes:

$$|D(j, o)| \geq \kappa \qquad \text{y} \qquad |V_j| > \mu.$$

Exigir derrotas en κ vecindarios distintos responde a la interacción entre ranuras: una opción que pierde con un vecindario puede ganar con otro, de modo que una sola observación no es evidencia suficiente. Con $\kappa = 2$ la opción debe perder en dos contextos diferentes, que en la práctica aparecen en pasadas o reinicios distintos. El mínimo $\mu = K/2 = 4$ garantiza que la poda nunca descarte más de la mitad de las opciones de una ranura.

Para cada poda se registra, por cada vecindario de $D(j, o)$, la opción ganadora, las precisiones y los conjuntos L y G; además, el multiconjunto de familias

$$\Phi(o) = \{\, v_{c_o}(i) : i \in L(o, b) \,\}$$

acumulado sobre esos vecindarios, que indica qué tipo de restricción rompe la opción, y un extracto de la salida del modelo en una instancia perdida. Esa información justifica la decisión y se guarda en poda.json; las derrotas que todavía no produjeron una poda se guardan en derrotas.csv. Las opciones candidatas se procesan en orden decreciente de pérdida neta acumulada, de modo que, si el mínimo μ impide podar todas, se eliminan primero las que más daño causan.

La comparación se hace instancia por instancia y no solo por precisión agregada. Dos opciones con la misma precisión pueden fallar instancias distintas; en ese caso la pérdida neta es cero y el criterio no las poda sin evidencia.

5.4. Pseudocódigo

Algoritmo 1 Ascenso por coordenadas con poda guiada por trazas.

Entrada: puntos iniciales $c_1, \ldots, c_R$; parámetros P, τ, κ, μ

Salida: mejor configuración observada y su precisión

$V_j \leftarrow \{0, \ldots, K-1\}$ para cada ranura j
para $r = 1$ **hasta** R **hacer**
    $c \leftarrow c_r$; $p \leftarrow \mathrm{Acc}(c)$
    para pasada = 1 **hasta** P **hacer**
        mejoró $\leftarrow$ falso
        para cada ranura j en (rol, estrategia, formato, estilo, cierre) **hacer**
            evaluar $\mathrm{Acc}(c')$ y las trazas de cada $c' \in N_j(c)$
            $b \leftarrow$ opción de mayor precisión; si empata con $c[j]$, $b \leftarrow c[j]$
            PODAR(j, b, c)
            si $\mathrm{Acc}(c \text{ con } j = b) > p$ **entonces** mejoró $\leftarrow$ verdadero
            $c[j] \leftarrow b$; $p \leftarrow \mathrm{Acc}(c)$
        si no mejoró **entonces** terminar el reinicio r
    guardar (c, p) como resultado del reinicio r
devolver el resultado de mayor precisión entre los R reinicios

Algoritmo 2 Procedimiento PODAR(j, b, c).

$u \leftarrow$ índices de c en las ranuras distintas de j
para cada opción $o \neq b$ evaluada en el paso **hacer**
    si $|L(o,b)| - |G(o,b)| \geq \tau$ **entonces** $D(j,o) \leftarrow D(j,o) \cup \{u\}$
$Q \leftarrow \{\, o \in V_j : o \neq b,\ |D(j,o)| \geq \kappa \,\}$, ordenado de mayor a menor pérdida neta acumulada
para cada $o \in Q$ **hacer**
    si $|V_j| \leq \mu$ **entonces** terminar
    $V_j \leftarrow V_j \setminus \{o\}$
    registrar o, la evidencia de cada vecindario de $D(j,o)$, $\Phi(o)$ y un extracto de la salida

Todas las evaluaciones pasan por una memoria (memo) indexada por la configuración. Si una configuración ya se evaluó en un paso, una pasada o un reinicio anteriores, se reutiliza su resultado sin una nueva consulta al oráculo.

5.5. Desempate, puntos iniciales y parada

Si varias opciones alcanzan la precisión máxima y la opción vigente está entre ellas, se conserva la vigente; en otro caso se elige la de menor índice. Así la configuración solo cambia cuando hay evidencia de mejora y el resultado no depende del orden de evaluación.

Los puntos iniciales son la configuración con todos los índices en 0 y $R - 1$ configuraciones generadas con random.Random(SEMILLA). Ninguno usa información previa sobre qué opción es preferible.

Un reinicio termina cuando una pasada completa no mejora la precisión o cuando se alcanzan P pasadas. La búsqueda termina al completar los R reinicios.

5.6. Propiedades

Proposición 1 (No decrecimiento)**.** Dentro de un reinicio, la precisión de la configuración vigente nunca disminuye.

Demostración. En cada paso, la configuración vigente pertenece a $N_j(c)$, porque $c[j]$ se incluye siempre en la vecindad, aunque haya sido podada. La opción elegida maximiza la precisión sobre ese conjunto, luego $\mathrm{Acc}(c \text{ con } j = b) \geq \mathrm{Acc}(c)$. Con temperatura 0.0 el oráculo es determinista y la precisión de c no cambia entre consultas. □

Proposición 2 (Terminación y cota de consultas)**.** Cada reinicio realiza a lo sumo $1 + P \cdot m \cdot K$ evaluaciones y la búsqueda completa a lo sumo $R\,(1 + P \cdot m \cdot K)$.

Demostración. Cada reinicio hace una evaluación inicial y como máximo P pasadas. En cada pasada, cada una de las m ranuras evalúa $|V_j \cup \{c[j]\}| \leq K$ configuraciones. Sumando, el reinicio realiza a lo sumo $1 + P \cdot m \cdot K$ evaluaciones, y la memoria solo puede reducir este número. Como R, P, m y K son finitos, el algoritmo termina. □

Proposición 3 (Mínimo de opciones)**.** En todo momento $|V_j| \geq \mu$ para cada ranura j.

Demostración. Inicialmente $|V_j| = K \geq \mu$. Podar solo elimina una opción cuando $|V_j| > \mu$, y elimina una por vez, por lo que después de cada eliminación $|V_j| \geq \mu$. □

Proposición 4 (Reproducibilidad)**.** Con los mismos datos, modelo, semilla y parámetros, dos ejecuciones producen la misma secuencia de configuraciones evaluadas y la misma configuración final.

Demostración. Con temperatura 0.0 la decodificación es voraz (greedy), así que la respuesta a cada par de configuración e instancia es fija. Los puntos iniciales dependen solo de SEMILLA, el orden de las ranuras es fijo y el desempate es determinista. Por inducción sobre los pasos, todas las decisiones coinciden. □

Las proposiciones no garantizan optimalidad global. El ascenso por coordenadas puede detenerse en una configuración en la que ningún cambio de una sola ranura mejora, aunque exista una mejor que exija cambiar dos ranuras a la vez. La poda agrega un segundo riesgo: una opción descartada en un contexto podría ser la mejor en otro, porque las ranuras interactúan. Los parámetros τ, κ y μ, y la reevaluación de la opción vigente aunque esté podada, limitan ese riesgo sin eliminarlo.

5.7. Análisis de complejidad

Se distinguen tres costos: el número de consultas al oráculo, el costo de cada consulta y el trabajo propio de la heurística.

Número de consultas

Sea Q el número de configuraciones distintas evaluadas. Por la proposición 2:

$$Q \le R\,(1 + P \cdot m \cdot K) = 3 \cdot (1 + 3 \cdot 5 \cdot 8) = 363.$$

Una vez que hay podas, cada pasada evalúa $\sum_j |V_j \cup \{c[j]\}|$ configuraciones, con $\mu \le |V_j| \le K$. Si todas las ranuras llegan a $\mu = 4$ opciones vivas, una pasada cuesta como máximo $m(\mu + 1) = 25$ evaluaciones en lugar de $m \cdot K = 40$.

Costo de una consulta

Una consulta genera una respuesta por instancia no guardada en caché y verifica sus restricciones. Si t_g es el tiempo de generar una respuesta de a lo sumo 384 tokens nuevos y t_v el de verificar una restricción, y cada instancia tiene a lo sumo 5 restricciones, el costo de una consulta es

$$O\big(n\,(t_g + 5\,t_v)\big) = O(n \cdot t_g),$$

porque la generación domina. El oráculo agrupa las peticiones en lotes de hasta 16 prompts, lo que reduce la constante pero no el orden.

Trabajo de la heurística

PODAR calcula $L(o, b)$ y $G(o, b)$ para a lo sumo $K - 1$ opciones con conjuntos de identificadores, en $O(n)$ cada una, y ordena a lo sumo K candidatas. La pérdida neta acumulada se suma sobre a lo sumo $R \cdot P$ vecindarios por opción. El costo por paso es $O(K \cdot n + K \cdot R \cdot P + K \log K) = O(K \cdot n)$, porque $R \cdot P \leq n$ con los valores del proyecto, y en toda la búsqueda $O(R \cdot P \cdot m \cdot K \cdot n)$, despreciable frente a la generación.

Costo total y comparación

El tiempo total es

$$T = O(Q \cdot n \cdot t_g) = O(R \cdot P \cdot m \cdot K \cdot n \cdot t_g),$$

polinomial en los parámetros. La enumeración exhaustiva evalúa las K^m configuraciones, con un costo $O(K^m \cdot n \cdot t_g)$, exponencial en m. La tabla 8 compara ambas cantidades con los valores del proyecto.

Tabla 8. Generaciones del modelo en el peor caso.

Método	Configuraciones	Generaciones (n = 100)
Enumeración exhaustiva (temperatura 0.0)	$8^5 = 32\,768$	3 276 800
Ascenso por coordenadas, $R = 3$, $P = 3$	≤ 363	$\leq 36\,300$
Ascenso con poda, pasadas 2 y 3 con $\mu = 4$	$\leq 3\,(1+40+2\cdot25) = 273$	$\leq 27\,300$

La última fila es una cota del caso en el que todas las ranuras alcanzan μ opciones vivas al final de la primera pasada de cada reinicio; el número real depende de cuántas podas ocurran y de cuántas configuraciones se repitan entre reinicios.

Memoria

La memoria guarda, por configuración evaluada, la precisión y las trazas de las instancias falladas, cada una con una salida de a lo sumo 384 tokens. El espacio es $O(Q \cdot n \cdot \ell)$, con ℓ la longitud máxima de una salida. La caché del oráculo en disco crece en la misma proporción y por eso se exporta comprimida.

5.8. Estructuras de datos

Tabla 9. Estructuras de datos del notebook de búsqueda.

Estructura	Tipo	Contenido y uso
memo	diccionario	Llave: tupla de los cinco índices y la temperatura. Valor: precisión, aciertos, diccionario de id a familia violada, contador de familias y trazas completas. Evita repetir consultas y permite comparar opciones instancia por instancia.
registro	lista de diccionarios	Una entrada por consulta nueva, con su contexto (reinicio, pasada, ranura, opción) y su resultado. Se exporta a evaluaciones.csv y registro_busqueda.json.
vivas	diccionario de listas	Conjunto V_j de opciones vivas de cada ranura.
derrotas	diccionario	Llave: (ranura, opción). Valor: diccionario de vecindario a evidencia (opción ganadora, precisiones, instancias perdidas y ganadas, familias). Implementa $D(j, o)$. Se exporta a derrotas.csv.
podas	lista de diccionarios	Decisiones de poda con su evidencia. Se exporta a poda.json.
historial	lista de reales	Precisión de cada consulta nueva, en orden, para la curva de búsqueda.

6. Procedimiento de ejecución

6.1. Notebook de búsqueda

1. Abrir Google Colab, elegir Archivo, Abrir notebook, GitHub, y pegar la dirección del repositorio. Seleccionar notebooks/01_busqueda.ipynb.

2. En Entorno de ejecución, Cambiar tipo de entorno de ejecución, seleccionar GPU T4.

3. Ejecutar la celda de la sección 1. Al terminar, elegir Entorno de ejecución, Reiniciar sesión.

4. Ejecutar la sección 2. Si USAR_DRIVE es verdadero, autorizar el acceso a Google Drive; la caché se guarda en MyDrive/ADA_Oraculo_G01/cache_oraculo.json. Si existe una caché de

una ejecución anterior con el mismo modelo y semilla, copiarla en esa ruta antes de ejecutar: las configuraciones ya generadas se resuelven sin usar la GPU.

5. Ejecutar las secciones 3 y 4. La sección 4 imprime una línea por consulta y una línea por cada poda. Con la GPU gratuita de Colab la búsqueda completa puede tardar varias horas.

6. Ejecutar las secciones 5 a 7: tablas de evaluaciones, análisis de trazas y validación con oraculo.validar sobre las 87 instancias reservadas.

7. Ejecutar la sección 8. Se escriben los archivos de resultados/, entrega.json y se descarga resultados_busqueda.zip. Descargar también el notebook con sus salidas (Archivo, Descargar, .ipynb) y subir todo al repositorio.

6.2. Notebook de evaluación

1. Si se cuenta con un archivo de prueba propio, subirlo al panel de archivos de Colab con el nombre datos_prueba.json, en /content. También se acepta una ruta en la variable de entorno ORACULO_DATOS o el archivo MyDrive/datos_prueba.json.

2. Abrir notebooks/02_evaluacion.ipynb, elegir GPU T4, ejecutar la sección 1 y reiniciar la sesión.

3. Ejecutar las secciones 2 a 4 sin modificar el código. La salida indica el origen de los datos, la precisión, el número de aciertos y las familias de restricción que más fallaron. Si no se encuentra datos_prueba.json, se evalúa sobre la partición de validación y la salida lo indica.

6.3. Reanudación después de una desconexión

Si Colab se desconecta durante la búsqueda, basta con abrir de nuevo el notebook, ejecutar la sección 1, reiniciar y ejecutar desde la sección 2. El oráculo carga la caché de Google Drive y las configuraciones ya generadas se resuelven sin usar la GPU. Por la proposición 4, la búsqueda recorre las mismas configuraciones en el mismo orden y continúa desde el punto donde se interrumpió.

6.4. Parámetros

Tabla 10. Parámetros de la búsqueda (sección 2 del notebook).

Parámetro	Valor	Efecto
GRUPO	"G01"	Identificador escrito en entrega.json
SEMANA	5	Semana académica escrita en entrega.json; debe confirmarse con la docente
MODELO	"qwen17b"	Modelo usado por el oráculo; cambiarlo invalida la caché
TEMPERATURA	0.0	Temperatura de todas las configuraciones evaluadas
SEMILLA	1	Semilla del oráculo y de los puntos iniciales aleatorios
MAX_PASADAS (P)	3	Pasadas máximas por reinicio
N_REINICIOS (R)	3	Número de puntos iniciales
UMBRAL_PODA (τ)	3	Pérdida neta mínima, en instancias, para registrar una derrota
CONTEXTOS_PODA (κ)	2	Vecindarios distintos con derrota necesarios para podar una opción
MIN_VIVAS (μ)	4	Opciones mínimas que conserva cada ranura
USAR_DRIVE	True	Guarda la caché en Google Drive

7. Interpretación de las salidas

Tabla 11. Salidas del sistema y su significado.

Salida	Significado
consulta k p mejor q ranura=o s	Consulta nueva número k: precisión p de la configuración evaluada, mejor precisión q hasta ese momento, opción probada y segundos empleados
poda ranura=o: derrotada en k contextos, pérdida neta x. . .	Se descartó la opción o porque perdió en k vecindarios distintos, con x instancias netas en total; se muestran las familias más frecuentes en las instancias perdidas
fin pasada p (reinicio r)	Precisión y configuración vigente al terminar la pasada
Resultado(precision=. . . , fallos=f/n)	Representación de un resultado del oráculo
validación: p (87 inst)	Precisión en la partición de validación seguida de las familias que más fallaron
consultas que habría hecho el ascenso sin poda	Cota de consultas sin poda para el mismo número de pasadas, para comparar con las realizadas
entrega.json	Configuración final: grupo, semana, índices de las cinco ranuras y temperatura

La precisión de búsqueda y la de validación miden cosas distintas. La primera se obtuvo sobre las mismas instancias usadas para elegir la configuración y tiende a sobrestimar el desempeño; la segunda se mide sobre instancias no usadas en la búsqueda. Ninguna de las dos sustituye la evaluación oficial de la docente sobre datos privados.

8. Verificación

8.1. Comprobaciones internas

- El notebook de búsqueda verifica que la partición de búsqueda tenga 100 instancias, la de validación 87 y que cada ranura tenga 8 opciones.
- El notebook de evaluación verifica que entrega.json contenga exactamente las cinco ranuras y la temperatura, que cada índice sea un entero entre 0 y 7 y que la temperatura sea un valor

permitido. Si alguna condición falla, se detiene con un mensaje que identifica el campo.

8.2. Prueba con oráculo sintético

El archivo pruebas/prueba_busqueda_simulada.py ejecuta las funciones del notebook de búsqueda sin GPU ni dependencias externas. Reemplaza el oráculo por uno sintético y determinista con una configuración óptima conocida: cada ranura distinta de la óptima hace fallar un subconjunto fijo de instancias. Se ejecuta desde la raíz del repositorio con el comando python pruebas/prueba_busqueda_simulada.py.

Tabla 12. Comprobaciones de la prueba sintética.

Comprobación	Propiedad verificada	Resultado
Se encuentra la configuración óptima y la precisión reportada es 100 %	Corrección en un caso con óptimo conocido	Correcta
Ninguna configuración se consulta dos veces	Funcionamiento de la memoria	Correcta
Cada consulta nueva queda en el registro	Trazabilidad	Correcta
Ningún reinicio termina por debajo de su punto inicial	Proposición 1	Correcta
Toda ranura conserva al menos μ opciones	Proposición 3	Correcta
Toda poda se apoya en derrotas en al menos κ vecindarios	Criterio de la sección 5.3	Correcta
Cada derrota tiene una pérdida neta de al menos τ instancias	Criterio de la sección 5.3	Correcta
Las consultas no superan 363	Proposición 2	Correcta

En la ejecución de la prueba, la búsqueda realizó 89 consultas y 16 podas, frente a la cota de 363 consultas sin poda. El programa termina con código 0 si todas las comprobaciones se cumplen y con código 1 en caso contrario.

8.3. Prueba mínima con el modelo

Para comprobar la instalación sin ejecutar la búsqueda completa, después de la sección 2 del notebook de búsqueda se puede evaluar una configuración sobre cinco instancias:

```
r = oraculo.evaluar(config, busqueda[:5], semilla=1)
```

El resultado es correcto si r.n es 5, r.precision está en $[0, 1]$, el número de trazas es $5 - 5 \cdot$ r.precision y cada traza tiene los campos id, violo y salida.

8.4. Reproducibilidad de los resultados

La configuración de entrega.json debe coincidir con la mejor configuración del notebook de búsqueda ejecutado y con el campo mejor_config de registro_busqueda.json. La precisión de búsqueda puede recalcularse a partir de evaluaciones.csv, y la caché comprimida permite repetir el análisis de trazas sin GPU.

9. Supuestos, limitaciones y trabajo pendiente

9.1. Supuestos

- La temperatura es 0.0 y la decodificación es voraz, de modo que el oráculo se comporta como una función determinista de la configuración y la instancia.
- La búsqueda usa solo las 100 instancias de búsqueda. La partición de validación no interviene en la elección de la configuración.
- Los datos privados de prueba tienen el formato de la sección 4.1 y sus familias de restricción están en el registro IFEvalG de open-instruct.

9.2. Limitaciones y errores conocidos

- La búsqueda es heurística y puede terminar en un óptimo local (sección 5.6).
- La poda puede descartar una opción que sería la mejor en otro contexto de las demás ranuras.
- Con $n = 100$ cada instancia representa un punto porcentual. Diferencias de una o dos instancias entre configuraciones pueden deberse a la muestra y no a una ventaja real.
- Una ejecución completa toma varias horas en una T4 gratuita y puede verse interrumpida por desconexiones de Colab.
- Si los datos de prueba incluyen familias que no están en IFEvalG, el oráculo lanza KeyError salvo que se le pasen registros adicionales mediante el parámetro registros de Oraculo.
- Si una instancia no trae el campo restriccion, la opción de cierre que repite las restricciones no agrega texto al prompt.

9.3. Componentes no desarrollados

- Estrategias de reparto del presupuesto de consultas (uniforme y adaptativo), Successive Halving y asignación por UCB1.
- Exploración de las temperaturas 0.3 y 0.7.
- Medición de tiempos con tamaños crecientes de n y comparación con la función de eficiencia teórica.

Referencias

Allen Institute for AI. (2025). *open-instruct* [Repositorio de software]. GitHub. https://github.com/allenai/open-instruct

Melo, D. (2026). *Proyecto-Oraculo-AyD* [Repositorio de software]. GitHub. https://github.com/DanielMelo404/Proyecto-Oraculo-AyD

Qwen Team. (2025). *Qwen3 technical report*. arXiv. https://arxiv.org/abs/2505.09388

Wright, S. J. (2015). Coordinate descent algorithms. *Mathematical Programming, 151*(1), 3–34. https://doi.org/10.1007/s10107-015-0892-3

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., y Hou, L. (2023). *Instruction-following evaluation for large language models*. arXiv. https://arxiv.org/abs/2311.07911